Strong Scaling of Flux.jl DDP and PyTorch

One Leonardo node · 4× A100-SXM-64GB · fixed global batch

Flux.jl DDP Thesis Research Group

2026-10-22

Leonardo EuroHPC / CINECA

Overview & Study Design

Executive Summary: What the Benchmark Shows

1 · Both Flux builds scale

Adding GPUs on a single node gives close-to-proportional throughput gains. Both Flux builds gain throughput from one to four GPUs.

2 · Flux is competitive with PyTorch

Throughput is close to PyTorch at every GPU count. Framework differences are smaller than the gain from doubling the GPU count.

3 · Guards cost throughput at two GPUs

The Flux build with runtime guards trails at two GPUs. At four GPUs it is still behind Flux (static), but it moves ahead of PyTorch and has the most efficient two-to-four-GPU transition of the three.

4 · The result is stable across repeats

Re-measuring every configuration in alternating order produced stable medians. One slow first repeat does not move the reported values.

Verdict

Strong scaling holds on this node: all three implementations convert additional GPUs into useful throughput, and the Flux builds are credible alternatives to PyTorch. The result is a **descriptive throughput comparison**. It says nothing about convergence, accuracy, or time-to-quality.

The Three Candidates: Two Flux Builds and PyTorch

All three candidates run the same model, data order, optimizer, and number of updates. Their main difference is how they synchronize gradients. *“Static” here means no runtime guards, not a static computation graph.*

Flux (static)

earlier build

The original Flux data-parallel implementation.

- One synchronization call per gradient array.
- No runtime checks around synchronization.
- Leanest communication path; this is the baseline for measuring the cost of the newer safety layer.

Flux (guarded)

current build

The Flux build with runtime guards.

- Same per-array synchronization as Flux (static).
- Adds a presence check: every GPU confirms it has started the update.
- Adds a finite-value check: non-finite gradients are rejected before the optimizer applies them.

PyTorch DDP

external reference

The comparison point outside Julia.

- Native distributed data parallel.
- Packs gradients into large buffers instead of one synchronization call per array.
- Overlaps communication with the backward pass; deterministic kernels are enabled.

What is held constant

Model weights, sample order, batch composition, optimizer schedule, and update count are identical in all runs. The intended contrast is the **synchronization layer**: guards in Flux, bucketing in PyTorch. The builds also differ in other implementation details.

Study Design: Fixed Work, More GPUs

Research question

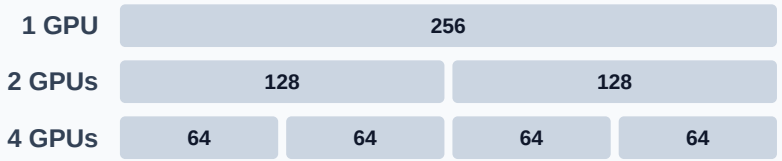
When the global batch is fixed, how does training throughput change as the same run moves from one to two and four GPUs? And how do the two Flux builds compare with PyTorch along the way?

This is **strong scaling**: the problem stays the same size while the hardware grows.

Platform: one Leonardo node with four A100-SXM-64GB GPUs connected by NVLink. Tiny ImageNet-200 ResNet-18, AdamW, global batch 256.

Scaling design

The global batch never changes. Each additional GPU receives a smaller slice of the same work, so every run performs the same 390 optimizer updates.



Measures

Throughput $Q_N = \frac{KB}{T_N}$ Speedup $S_N = \frac{T_1}{T_N}$ Efficiency $E_N = \frac{S_N}{N}$

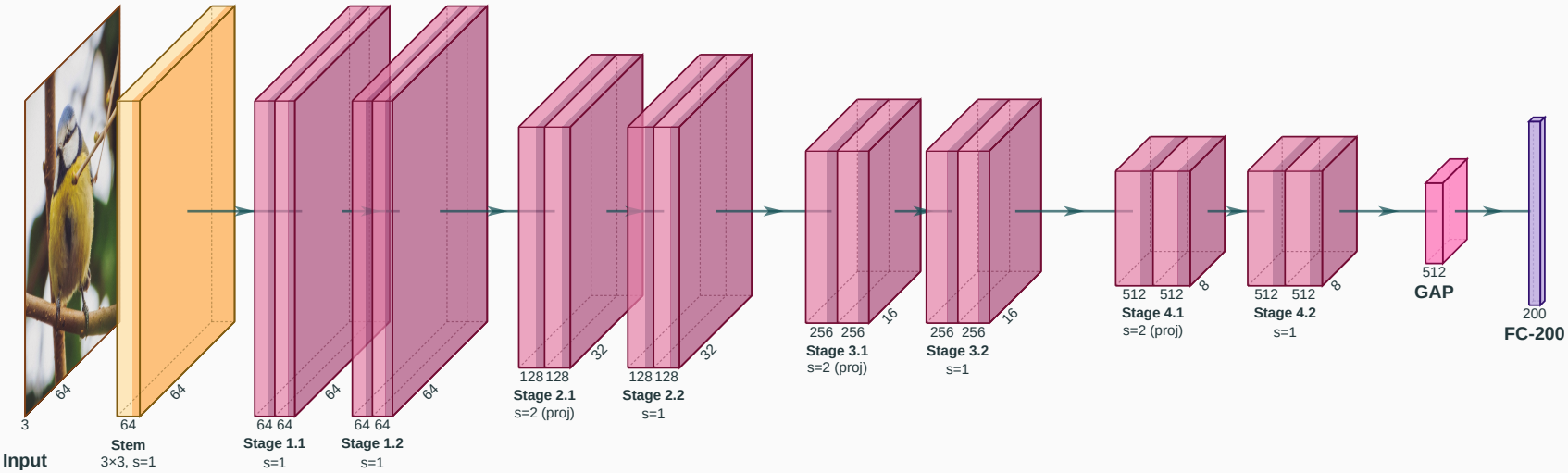
$K = 390$ updates, $B = 256$ images, T_N = epoch time on N GPUs.

Scope: fixed-work throughput only. No validation loss, accuracy, or convergence claim.

Model & Workload

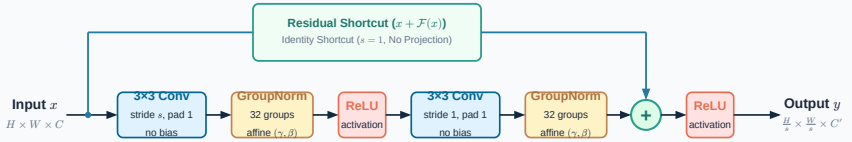
The Model at a Glance

A ResNet-18 adapted to 64× 64 inputs. GroupNorm computes its statistics within each image, so the local batch size does not change the normalization.



11.27M	62	20 Layers	64×64×3
Trainable Parameters	Parameter Arrays	GroupNorm(32)	Input Resolution
11,271,432 exact count	20 Conv + 40 GN + 2 Dense	Batch-size invariant (no sync)	Tiny ImageNet (200 classes)

Residual block



- Two 3× 3 convolutions with GroupNorm and ReLU, plus a skip connection.
- All convolutions are bias-free; normalization carries the shift.
- GroupNorm-32 computes statistics within each image, so the output for an image does not depend on the other images on that GPU.

Where the 62 parameter arrays come from

Component	Specification & Layers	Output Shape	Parameters	Arrays
Stem	3×3 Conv (s1, p1, no bias) + GroupNorm(32) + ReLU	$64 \times 64 \times 64$	1,728	3
Stage 1	2 BasicBlocks (64 ch, stride 1, no projection)	$64 \times 64 \times 64$	147,456	12
Stage 2	2 BasicBlocks (128 ch, stride 2, 1×1 proj + GN)	$32 \times 32 \times 128$	524,288	15
Stage 3	2 BasicBlocks (256 ch, stride 2, 1×1 proj + GN)	$16 \times 16 \times 256$	2,097,152	15
Stage 4	2 BasicBlocks (512 ch, stride 2, 1×1 proj + GN)	$8 \times 8 \times 512$	8,388,608	15
Pooling	Global spatial average pooling (GAP)	512	0	0
Classifier	Dense Linear ($512 \rightarrow 200$ logits) + bias	200 logits	102,600	2
Total	**ResNet-18 GN-32 (20 Conv + 20 GN + 1 Dense)**	**200 classes**	**11,271,432**	**62**

Workload, Batching & Optimizer

Data and batches

- **Dataset:** Tiny ImageNet-200, 100,000 training images.
- **Effective epoch:** 99,840 images, after dropping a 160-image tail so the batch divides exactly at one, two, and four GPUs.
- **Global batch:** 256 images per optimizer update, always.
- **Local batch:** 256 on one GPU, 128 on two, 64 on four.
- **Measured work:** 390 updates per epoch.

Optimizer

- **AdamW** with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, learning rate 10^{-3}.
- Weight decay applies to convolution and linear weights; biases and normalization parameters are excluded.
- A linear learning-rate ramp followed by cosine decay. The measured epoch sits inside the ramp, so the study compares throughput, not converged quality.

Fixed-work rule

Every candidate and every GPU count sees the same sample permutation, the same deterministic transforms, the same initial weights, and exactly 390 gradient updates.

Measurement Protocol

How a Run Is Timed

One run

- **Reset to** the same initial weights and optimizer state.
- Run 20 untimed warm-up updates. Compilation and kernel selection happen here, before the timer.
- **Time** 390 updates over one full epoch.
- **Wait for all GPUs to finish** before stopping the timer, so the recorded duration covers every process.

The repeat design

- Every candidate is measured at one, two, and four GPUs.
- **Five repeats** per configuration.
- Repeat order **alternates**: one-to-four GPUs, then four-to-one, so slow drift is spread across candidates rather than concentrated in one.
- The reported value is the **median** of the five repeats. A single slow start does not dominate the result.

Platform

One Leonardo node with four A100-SXM-64GB GPUs connected by NVLink. Flux runs through MPI; PyTorch runs through torchrun. Single-GPU runs use the same code path without any distributed communication.

Memory limit

Before timing, each configuration must stay below 80% of device memory during a short trial. Every configuration passed at global batch 256. A larger batch exceeded the margin and was not used, so all candidates run under the same memory constraint.

Empirical Results & Scaling Analysis

All Measurements at a Glance

All times are for one epoch of 390 updates at global batch 256.

Candidate	1 GPU (s)	2 GPUs (s)	4 GPUs (s)	1 GPU img/s	2 GPUs img/s	4 GPUs img/s	Speedup (4 GPUs)	Efficiency (4 GPUs)
Flux (static)	91.09	47.46	25.53	1,096.1	2,103.8	**3,910.8**	**3.568x**	**89.20%**
Flux (guarded)	91.10	52.21	27.92	1,095.9	1,912.2	3,575.5	3.263x	81.57%
PyTorch DDP	95.74	49.79	28.59	1,042.9	2,005.3	3,492.0	3.348x	83.71%

One GPU

Both Flux builds run at the same speed. Both are slightly faster than PyTorch before any communication begins.

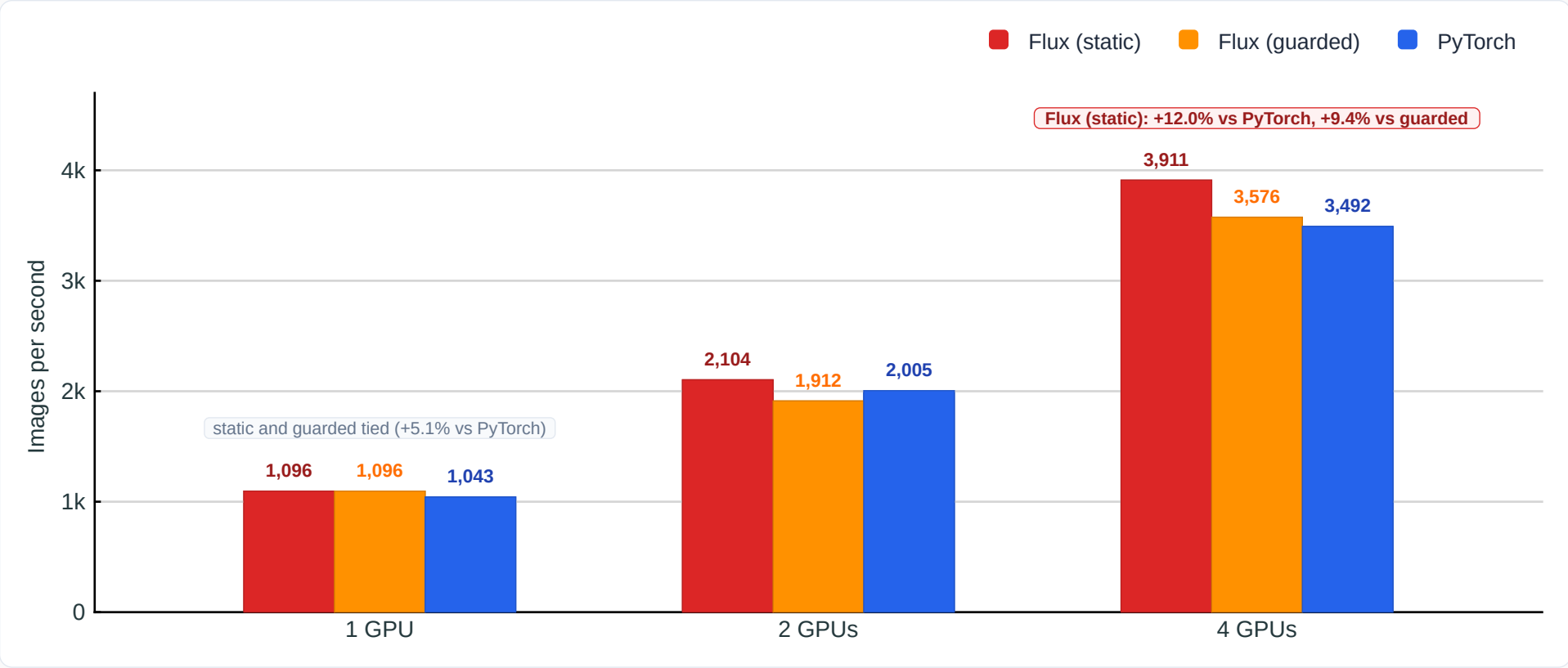
Two GPUs

Flux (static) has the highest throughput. PyTorch and Flux (static) are effectively tied on speedup; Flux (guarded) is behind, consistent with extra work per update.

Four GPUs

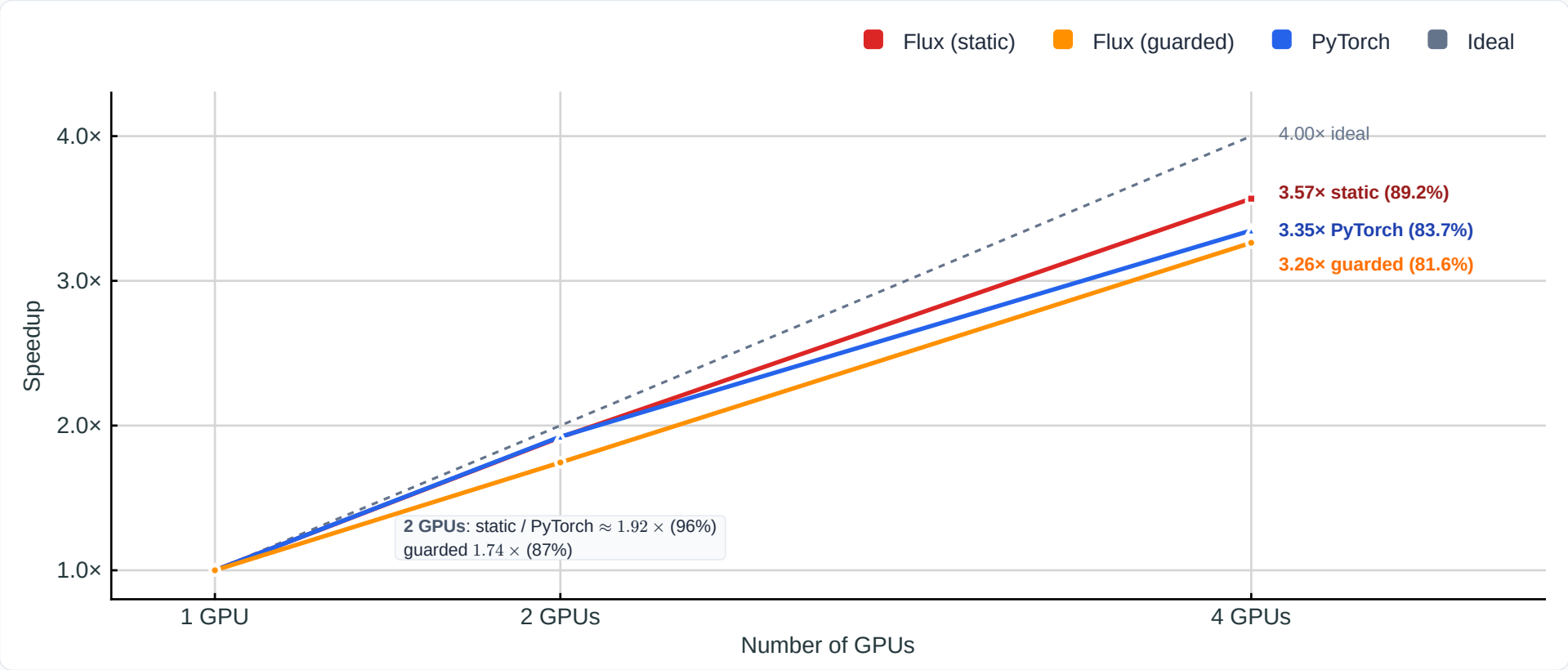
Flux (static) leads overall. Both Flux builds overtake PyTorch, and the guarded build nearly closes its two-GPU gap.

Absolute Throughput



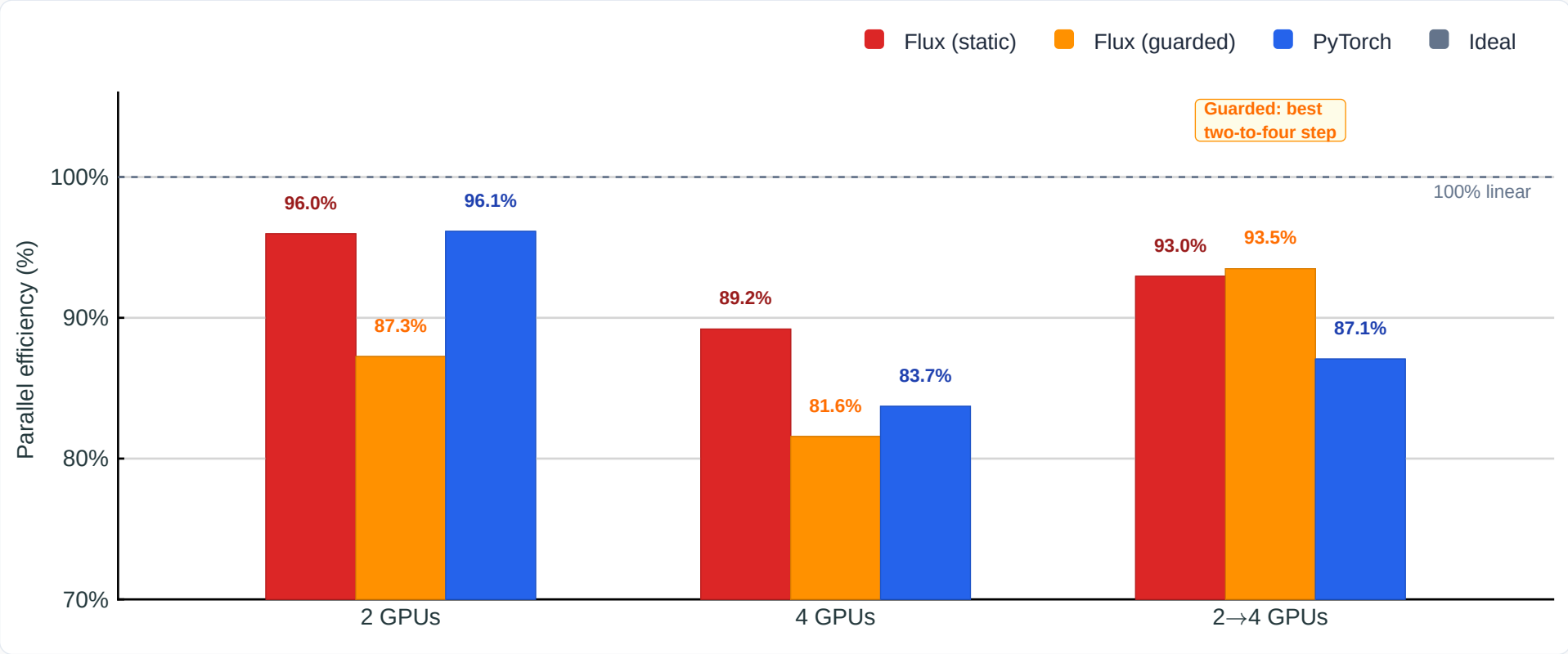
Takeaway: Flux is competitive at every GPU count, and the guarded build finishes ahead of PyTorch at four GPUs. Flux (static) leads overall.

Strong Scaling Speedup



Takeaway: Flux (static) has the highest overall speedup. PyTorch is marginally ahead at two GPUs, but the gap is 0.2% and disappears at four. Speedup is relative to each candidate's own one-GPU time.

Parallel Efficiency



Takeaway: Overall efficiency falls as GPUs are added, for every candidate. Flux (guarded) has the strongest step from two to four GPUs; PyTorch drops the most.

Comparisons by GPU Count

One GPU — reference

1,096 (Flux) **1,043** (PyTorch)

- The two Flux builds are indistinguishable.
- Both Flux builds are slightly faster than PyTorch before communication.

Two GPUs — mixed

2,104 (static) **2,005** (PyTorch) **1,912** (guarded)

- Flux (static) leads.
- PyTorch scales better than Flux (guarded) at this size.

Four GPUs — peak

3,911 (static) **3,576** (guarded) **3,492** (PyTorch)

- Both Flux builds beat PyTorch.
- Flux (static) has the largest margin; Flux (guarded) is just ahead.

The shape of the result

On one GPU the frameworks are close. PyTorch is strongest at two GPUs. At four GPUs, both Flux builds overtake it: the guarded build moves from behind PyTorch at two GPUs to ahead at four.

Incremental Scaling: Two to Four GPUs

The two-to-four-GPU transition

Efficiency at four GPUs mixes two effects: the quality of the step from one to two, and the quality of the step from two to four. The second transition is the better test of how a candidate behaves once distribution is already in place.

$$E_{2\to4} = \frac{\frac{T_2}{T_4}}{2}$$

Flux (guarded) has the weakest first transition but the **strongest second**: its two-to-four-GPU step is the most efficient, even though it remains behind Flux (static) overall.

93.5%

Flux (guarded)

highest step efficiency

93.0%

Flux (static)

close second

87.1%

PyTorch DDP

lowest step efficiency

Interpretation

Flux (static) remains the strongest overall. Flux (guarded) is the most efficient when scaling from two to four GPUs, consistent with a fixed per-update cost being spread over more hardware. The benchmark is descriptive and does not establish that mechanism.

Technical Deep Dives

The First-Repeat Outlier

What happened on the first repeat

one-off

- Flux (guarded), single GPU:
  - First repeat: **680.8 s**
  - Repeats two to five: 91.0–91.1 s
- The first repeat is about 7.5 times slower. Later repeats agree closely, within a tenth of a second.
- The five-repeat **median** is 91.1 s. Dropping the first repeat changes it by less than 0.02%, so the reported median is not sensitive to that run.

Why the median, not the mean

method

- The mean is dragged upward by the first repeat.
- A mean-based speedup would imply **more than linear scaling** on two and four GPUs, which is not plausible for this workload.
- The median is the more representative summary when one run is disturbed by an external event.

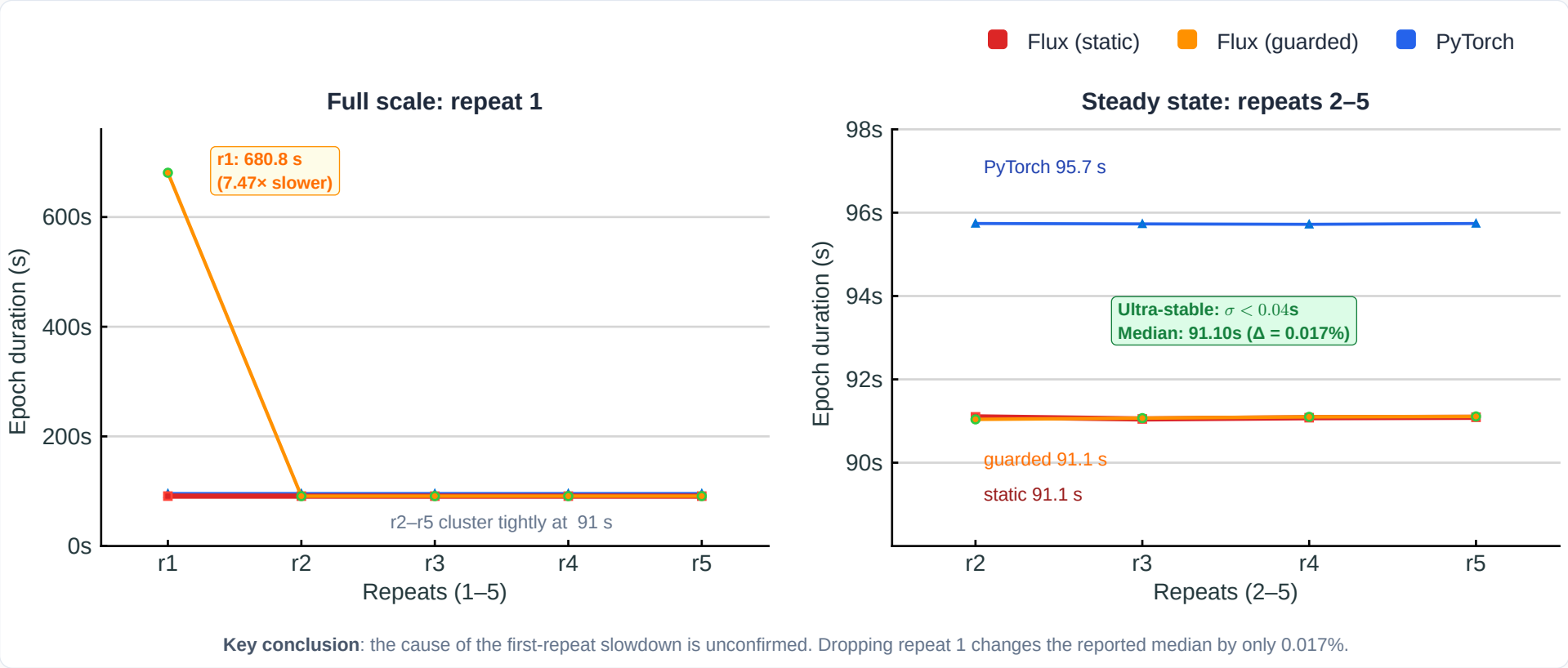
Cause: still unconfirmed

The most plausible explanation is a cold file-cache load of the training data on the first read. No page-fault, disk, or cache telemetry was recorded, so this remains a hypothesis rather than a claim.

It is not compilation time

Twenty warm-up updates run before the timer starts. Julia compilation and GPU kernel selection happen there. The timer starts only after the weights and optimizer state are reset.

Repeat Consistency



Key conclusion: the cause of the first-repeat slowdown is unconfirmed. Dropping repeat 1 changes the reported median by only 0.017%.

Takeaway: Repeats two to five agree closely for all three candidates. The median removes the single cold-start event without discarding any run.

Guard Cost and Synchronization Strategy

Flux (guarded) — 64 calls

- One presence check, so no GPU starts an update alone.
- One gradient all-reduce per parameter array: 62 calls.
- One finite-value check that rejects non-finite gradients.
- 64 synchronization calls per update, none of them overlapped with computation.

Flux (static) — 62 calls

- One gradient all-reduce per parameter array: 62 calls.
- No presence check and no finite-value check.
- The leanest path, and the baseline for the cost of the two extra checks.

PyTorch DDP — bucketed

- Gradients are packed into large contiguous buffers.
- Fewer, larger synchronization calls.
- Communication overlaps the backward pass instead of following it.

What the numbers show

Flux (static) is about 9% faster than Flux (guarded) at two and four GPUs. The gap is consistent with per-update guard work, but the two Flux builds also differ in other implementation details.

What this does not establish

The throughput difference cannot be attributed to the two extra checks alone. The benchmark compares two builds, not two isolated lines of code.

Numerical Reproducibility: What It Does and Does Not Support

What was compared

- Parameter snapshots after the same 390 updates, with identical data, initial weights, and learning-rate schedule.
- 62 parameter arrays per model, about 11.27 million values.
- Six comparisons between GPU counts within a framework; nine across frameworks.
- Within each distributed run, all ranks agree on the final state; the differences are between separate runs.

What the snapshots show

- **Weights:** about 2% relative L2, the same order as the difference between GPU counts of one framework.
- **AdamW moments:** 6–9% for the second moment and 18–30% for the first, as expected for state accumulated over all updates.
- **Schedule counters:** exact match.
- **Low-magnitude leaves:** one GroupNorm bias reaches 37% on an absolute difference of 0.003, because its reference norm is close to zero.

Why Flux repeats differ

- Five repeats per configuration: Flux gives five distinct final-state hashes at every GPU count; PyTorch gives one.
- Fresh processes can select numerically different cuDNN convolution algorithms: selection is timing-based and has no determinism filter. This happens at one GPU too, where there are no collectives at all.
- PyTorch requires deterministic algorithms and disables autotuning.

What that means for the comparison

Run-to-run variation is the same order as the differences between GPU counts and between frameworks. With one snapshot per configuration it cannot be separated from them. The comparison is descriptive; it is not evidence that one candidate computes the wrong answer.

Conclusions & Next Steps

What the Benchmark Supports

Findings

- **Strong scaling holds on this node.** Both Flux builds turn additional GPUs into useful throughput and stay competitive with PyTorch.
- **Flux (static) is the throughput leader at four GPUs.** Flux (guarded) remains behind it, but has the best two-to-four-GPU step efficiency.
- **The result is not an artifact of run order.** Re-measuring every configuration in alternating order produced stable medians. Dropping the one anomalous first repeat changed the affected median by less than 0.02%.

What it does not claim

- **No convergence claim.** The measured epoch sits inside the learning-rate ramp; there is no validation accuracy or time-to-quality result.
- **No claim of numerical equivalence.** Aggregate weight difference is about 2% relative L2; individual arrays can differ more.
- **No causal claim about the guards.** The two Flux builds differ in more than the two guard calls.
- **No generalization beyond this node.** One node with four GPUs.

Bottom line

On this node, for this model and batch size, Flux is a **better** alternative than PyTorch. The benchmark does not separate framework effects from GPU-count effects. I still wouldn’t merge the current version upstream, because the conditional graph overhead still has an impact (which should be zero in a static graph).

Next Steps

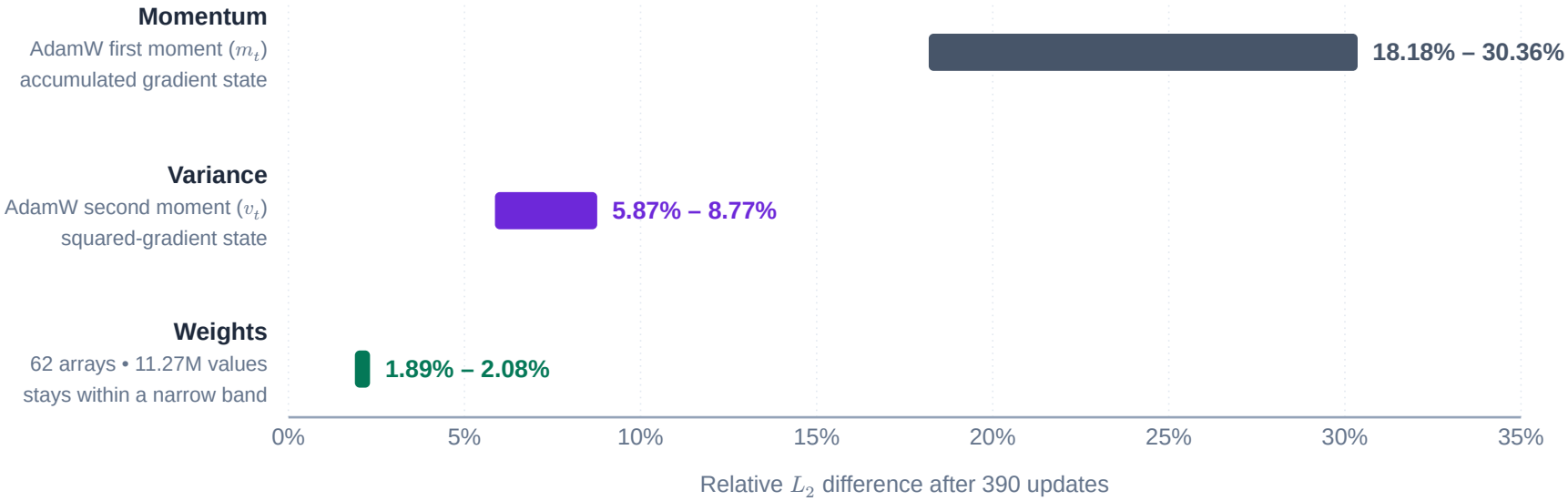
Reduce synchronization overhead

- **Gradient bucketing in Flux.** Pack small arrays into large contiguous buffers to cut the number of synchronization calls.
- **Overlapped backward communication.** Start synchronization while the backward pass is still running.
- Both changes reduce synchronization work per update.

Extend the evidence

- **Cold-cache experiment.** Repeat the first measurement with page-fault and disk telemetry to confirm or reject the cache hypothesis.
- **Deterministic-mode sensitivity.** Repeat the comparison with cuDNN algorithm selection restricted to a deterministic set, or with a fixed algorithm, to separate framework differences from run-to-run variation.
- **Multi-node scaling.** Move from NVLink to the InfiniBand fabric.

Backup: Diagnostic Snapshot Differences



Key observation: cross-framework weight differences ($\approx 2.0\%$) are comparable to differences between GPU counts within one framework ($\approx 2.0\%$). Schedule counters match exactly (0.0%), consistent with identical update counts.

Takeaway: Schedule counters match exactly, weights stay within a narrow band, and optimizer moments diverge more, as expected for accumulated state. A few low-magnitude arrays can look large in relative terms.